

Jordan Isaac

CS499

6/7/2026

Milestone Four Enhancement Three: Databases

Artifact: CS465 Travlr Getaways

Artifact Description

The artifact selected for enhancement is the Travlr Getaways full-stack web application originally developed in CS 465: Full Stack Development I. Travlr Getaways is a travel booking and management application that allows users to view, create, update, and delete travel packages through a web-based interface. The application uses Angular for the client-side interface, Node.js and Express for the server-side application logic, and MongoDB with Mongoose for database management.

The database component of the application stores trip information including trip codes, trip names, trip durations, resort locations, pricing information, images, and descriptive content. The original implementation provided basic CRUD functionality but lacked advanced validation, indexing, and structured database error handling. While the application successfully stored and retrieved information, it did not fully enforce data integrity requirements or optimize query performance for future growth.

The database enhancement focused on strengthening the data layer through improved schema validation, optimized database indexing, and enhanced error handling. These modifications transformed the database implementation from a basic storage mechanism into a

more robust, scalable, and reliable solution capable of supporting real-world application requirements. The enhancements demonstrate the application of database design principles and best practices that are commonly used in modern enterprise software development environments.

Justification for Inclusion in the ePortfolio

This artifact was selected for inclusion in the ePortfolio because it demonstrates my ability to design, implement, evaluate, and improve database-driven applications using industry-standard technologies. Database systems serve as the foundation for most modern software applications, making database design and management an essential skill for software engineers, backend developers, cybersecurity professionals, and cloud engineers.

The original artifact provided an opportunity to showcase foundational database functionality through CRUD operations and data persistence. However, the enhancement process allowed me to demonstrate a higher level of technical competency by applying advanced database concepts such as schema validation, indexing strategies, and structured error handling. These enhancements moved the application beyond basic functionality and introduced features commonly found in production-level systems.

Additionally, this artifact demonstrates my ability to evaluate existing software, identify weaknesses, and implement meaningful improvements that increase reliability, maintainability, performance, and security. The enhancement process required critical thinking, problem solving, testing, and troubleshooting, all of which are important skills within the field of computer science.

The artifact aligns closely with my professional interests in software engineering, backend development, cloud technologies, and cybersecurity. Organizations increasingly depend on scalable and reliable database systems to support business operations, making the knowledge and skills demonstrated within this enhancement highly relevant to my future career goals.

Course Outcomes Achieved Through the Enhancement

The database enhancement demonstrates achievement of several outcomes within the Computer Science program.

Outcome 1: Employ strategies for building collaborative environments that enable diverse audiences to support organizational decision making in the field of computer science.

Reliable data is critical for organizational decision making. By implementing schema validation and data integrity controls, the enhanced database helps ensure that users and stakeholders can trust the information stored within the system. Accurate data collection and management support informed decision making and improve overall system reliability.

Outcome 2: Design, develop, and deliver professional-quality oral, written, and visual communications that are coherent, technically sound, and appropriately adapted to specific audiences and contexts.

The enhancement required extensive documentation, testing, and explanation of technical decisions. Throughout the project, I documented the purpose of each enhancement, explained implementation strategies, and communicated testing results through written narratives and

visual evidence. These activities demonstrate the ability to communicate complex technical concepts effectively.

Outcome 3: Design and evaluate computing solutions that solve a given problem using algorithmic principles and computer science practices and standards appropriate to their context.

The enhancement process required evaluation of the existing database design and identification of areas for improvement. Validation rules, indexing strategies, and structured error handling were implemented to address limitations in the original design. These modifications improved both the functionality and efficiency of the system while adhering to industry best practices.

Outcome 4: Demonstrate an ability to use well-founded and innovative techniques, skills, and tools in computing practices.

Modern database technologies including MongoDB and Mongoose were utilized to implement advanced validation mechanisms, indexing strategies, and data management practices. These technologies provide scalable solutions for handling structured and semi-structured data within modern web applications.

Outcome 5: Develop a security mindset that anticipates adversarial exploits in software architecture and design.

Input validation and data integrity controls represent important security practices within software development. The implementation of validation constraints and unique identifiers reduces opportunities for invalid or malicious data to enter the database. Structured error

handling also prevents exposure of unnecessary database information while improving system reliability.

Enhancement Narrative

The primary objective of this enhancement was to improve the overall quality, reliability, and performance of the database layer within the Travlr Getaways application. Three major database enhancements were implemented to achieve this objective.

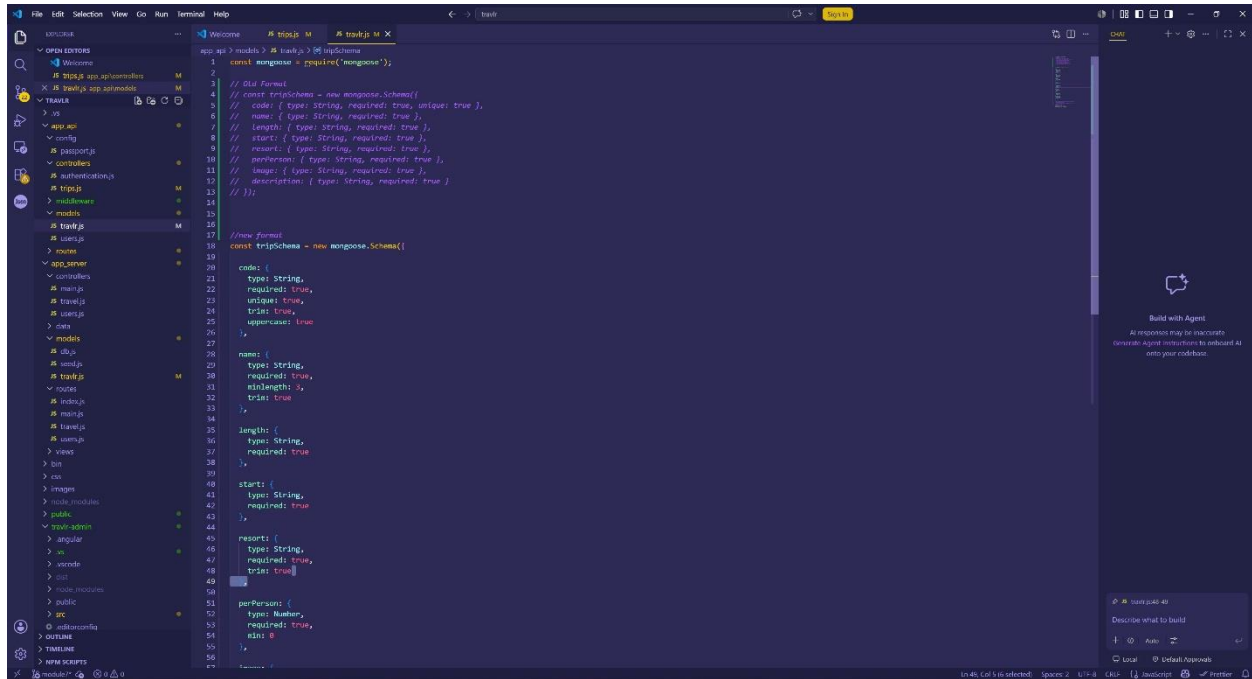
The first enhancement focused on strengthening schema validation. The original database schema primarily relied on required field validation. Additional validation rules were introduced including minimum character lengths, numeric range validation, automatic trimming of whitespace, uppercase formatting for trip codes, and unique constraints for trip identification. These improvements help maintain data consistency while preventing invalid records from being stored within the database.

The second enhancement focused on improving database performance through indexing. Indexes were added to fields that are commonly used during search operations, including trip names and resort locations. Database indexes allow MongoDB to locate records more efficiently and reduce the amount of collection scanning required to satisfy user queries. Although the current dataset is relatively small, these enhancements improve scalability and demonstrate best practices for database optimization.

The third enhancement involved implementing structured database error handling. Prior to the enhancement, database exceptions were not consistently managed or communicated to the user. Custom error handling was added to identify validation failures, duplicate key violations,

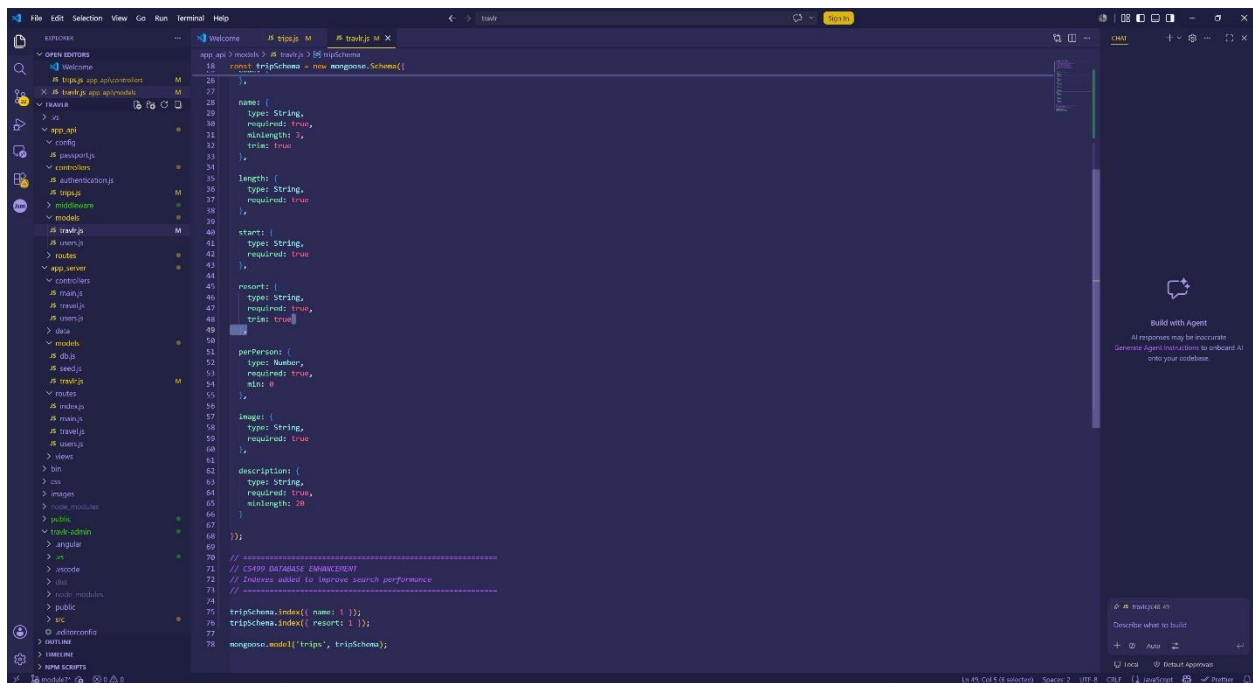
and unexpected database exceptions. These improvements provide more meaningful feedback to users while simplifying troubleshooting and maintenance activities.

Figure 1. Enhanced MongoDB Schema Validation



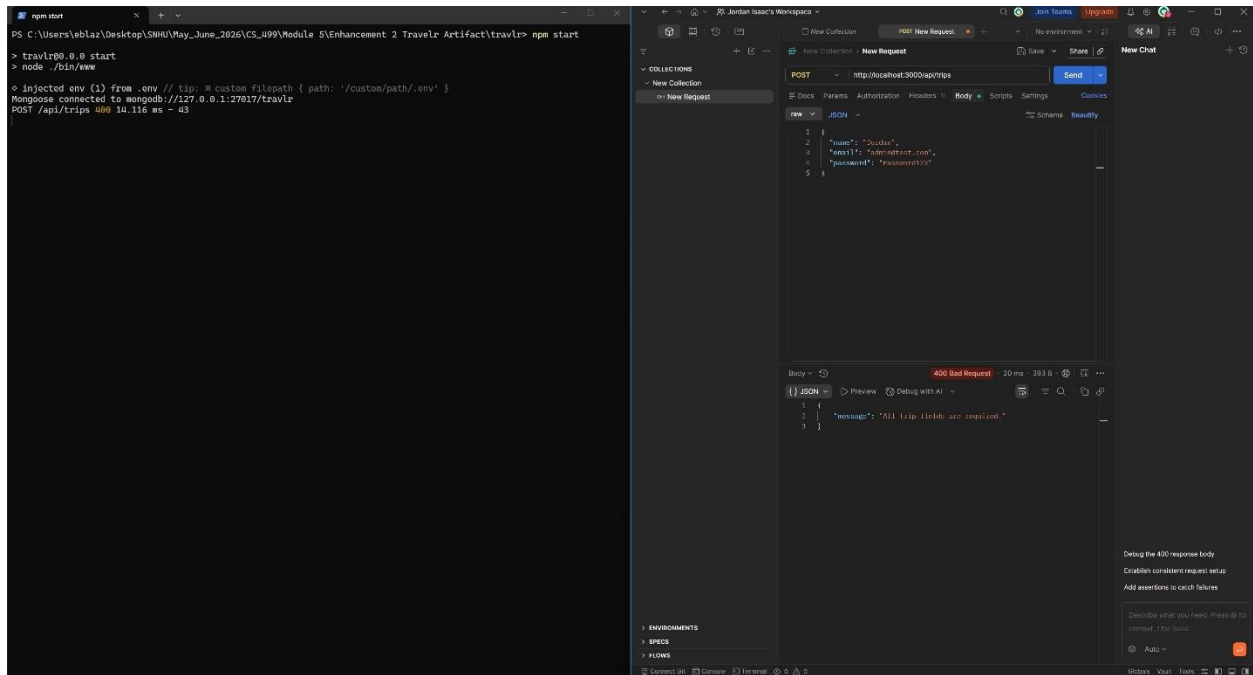
This screenshot displays the enhanced MongoDB schema validation implemented within the Trip model. Validation rules were added using required fields, minimum length constraints, minimum numeric values, uppercase formatting, and trimming functionality. These enhancements improve data integrity by preventing invalid records from being stored in the database and ensuring consistent formatting of trip information.

Figure 2. Database Index Optimization



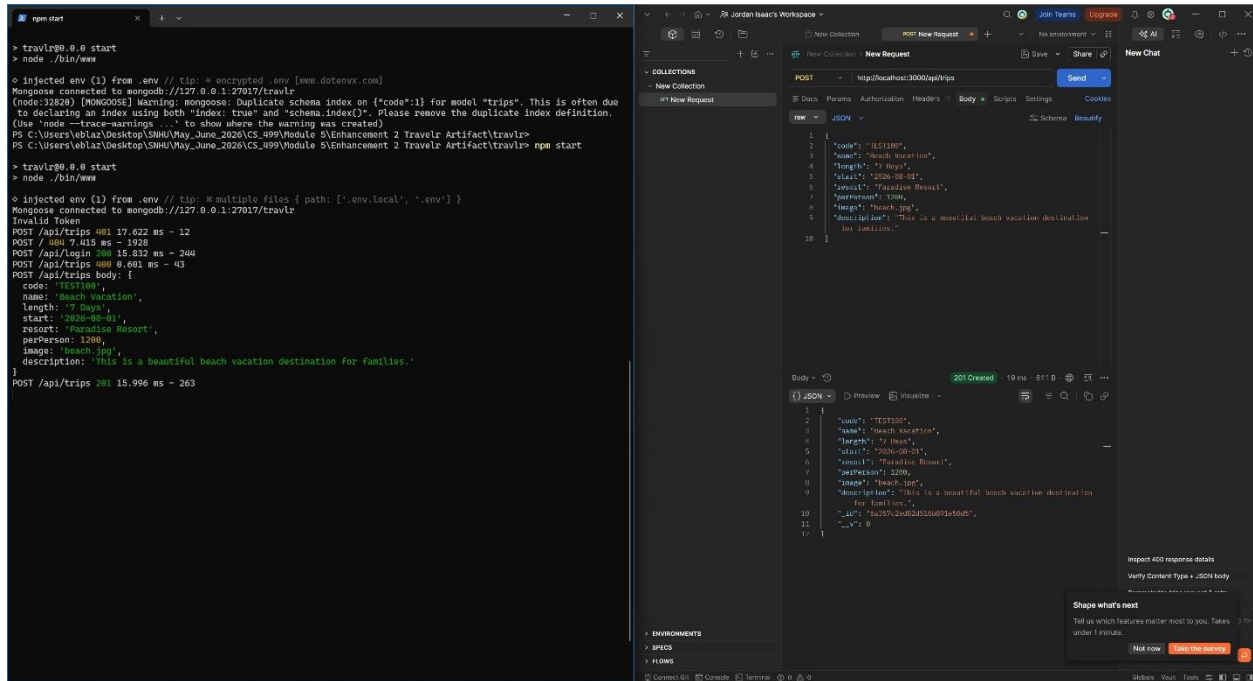
This screenshot displays the database indexing enhancements implemented within the MongoDB schema. Indexes were added to the name and resort fields to improve query performance during search operations. Database indexes reduce the amount of collection scanning required during retrieval operations and improve overall scalability as the number of trip records increases.

Figure 3. Validation Error Testing



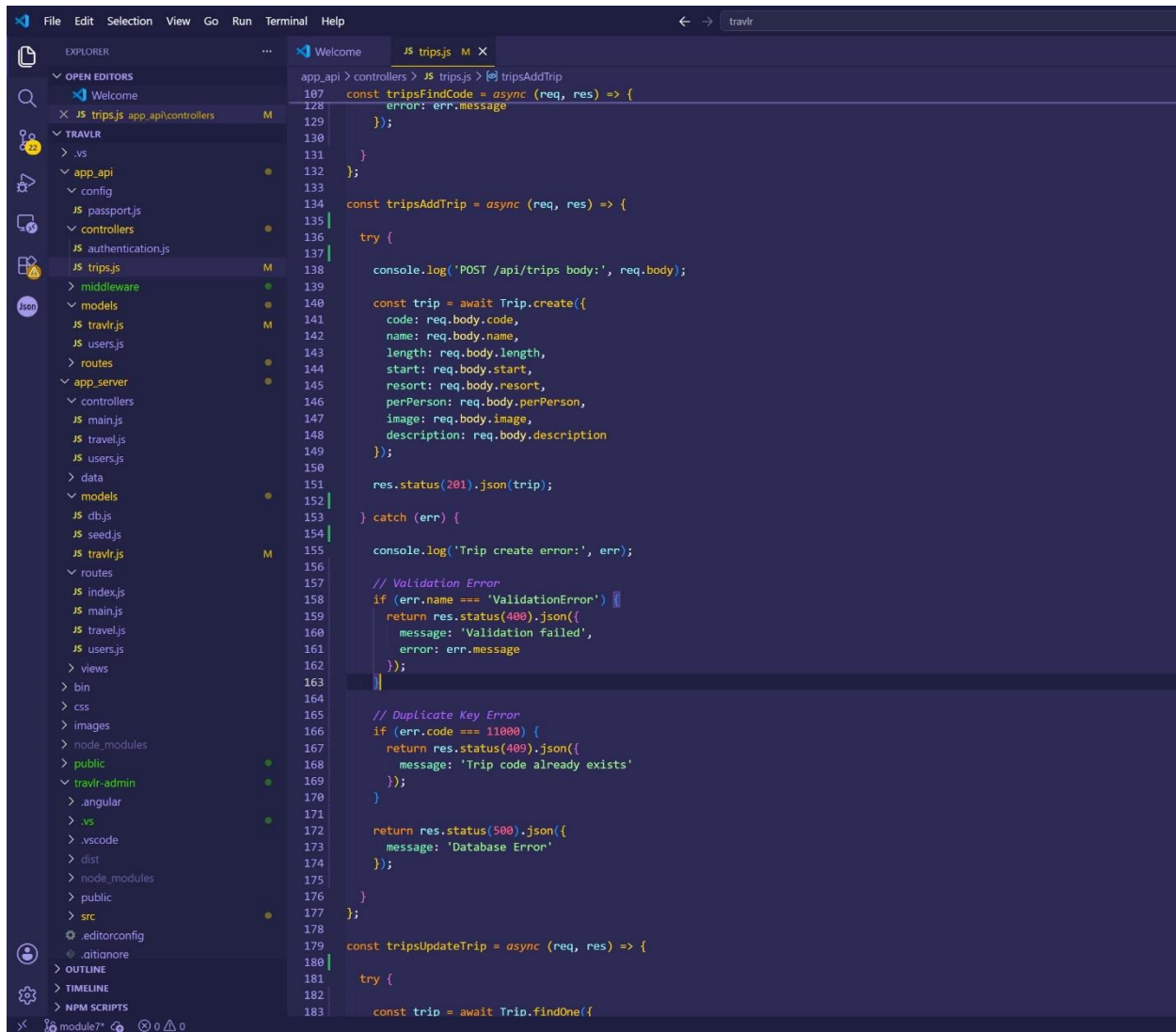
This screenshot demonstrates server-side validation preventing incomplete records from being inserted into the database. The application verifies that all required fields are provided before attempting database operations. This enhancement improves reliability by preventing invalid data from entering the MongoDB collection.

Figure 4. Successful Database Insert



This screenshot demonstrates successful insertion of a trip document after validation requirements have been satisfied. The application processes the request, validates the submitted data, and stores the record within MongoDB. This confirms proper integration between the Angular frontend, Express API, and MongoDB database.

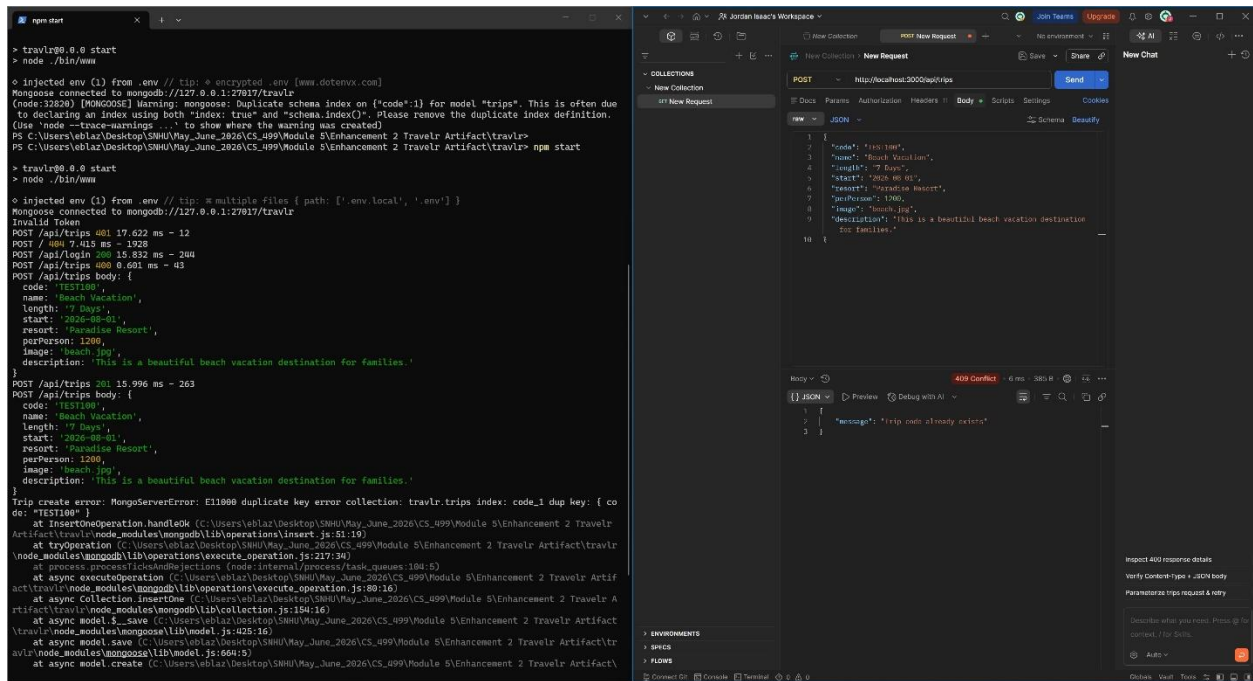
Figure 5. Duplicate Record Protection



```
107 const tripsFindCode = async (req, res) => {
128   error: err.message
129 });
130 }
131 }
132 };
133
134 const tripsAddTrip = async (req, res) => {
135   try {
136     console.log('POST /api/trips body:', req.body);
137
138     const trip = await Trip.create({
139       code: req.body.code,
140       name: req.body.name,
141       length: req.body.length,
142       start: req.body.start,
143       resort: req.body.resort,
144       perPerson: req.body.perPerson,
145       image: req.body.image,
146       description: req.body.description
147     });
148     res.status(201).json(trip);
149   } catch (err) {
150     // Validation Error
151     if (err.name === 'ValidationError') {
152       return res.status(400).json({
153         message: 'Validation failed',
154         error: err.message
155       });
156     }
157     // Duplicate Key Error
158     if (err.code === 11000) {
159       return res.status(409).json({
160         message: 'Trip code already exists'
161       });
162     }
163     return res.status(500).json({
164       message: 'Database Error'
165     });
166   }
167 }
168
169 const tripsUpdateTrip = async (req, res) => {
170   try {
171     const trip = await Trip.findOne({
```

This screenshot displays the enhanced database error handling logic implemented within the Express controller. Specific responses are returned for validation failures, duplicate key violations, and unexpected database errors. This enhancement improves system maintainability by providing meaningful feedback to users and simplifying troubleshooting for administrators.

Figure 6. Enhanced Database Error Handling



This screenshot demonstrates duplicate record protection implemented through MongoDB unique constraints. The application prevents duplicate trip codes from being stored and returns an informative error message to the user. This enhancement improves database integrity by enforcing uniqueness requirements across trip records.

Reflection

The database enhancement process provided valuable experience in evaluating and improving an existing software solution. While the original application functioned correctly, the enhancement process highlighted the importance of designing systems that not only work but also remain reliable, scalable, maintainable, and secure.

One of the most valuable lessons learned during this enhancement was the importance of data integrity. Database systems often serve as the central source of truth within software applications, making validation and consistency critical design considerations. Implementing schema validation demonstrated how small improvements can significantly reduce the likelihood of invalid or inconsistent data entering the system.

The enhancement also reinforced the value of performance optimization. Although the application currently manages a relatively small dataset, the addition of indexes demonstrates proactive planning for future growth. This experience helped me better understand how database design decisions can directly impact application responsiveness and scalability.

Overall, this enhancement strengthened my understanding of database management principles and provided practical experience working with MongoDB and Mongoose in a full-stack application environment. The knowledge gained through this project will be valuable in future software engineering, cloud computing, and cybersecurity roles where reliable and scalable data management solutions are essential.